

- Making Two Cameras Agree: EO/IR Fusion & Self-Calibration
 - 0. The 30-second version
 - 1. Vocabulary (read this first)
 - 2. The setup and what each piece did
 - 3. How "do these two boxes match?" is measured: IoU
 - A real example from our footage (frame 0)
 - 4. Why it was failing: the cameras don't share a coordinate system
 - The trap we ruled out
 - 5. The fix: one fixed "translation" between the cameras (registration)
 - Why one fixed formula is even valid
 - 6. Results after registration
 - 7. The clever part: it calibrates itself
 - The practical recipe for a fixed installation
 - 8. Honest limitations (what to watch for)
 - 9. One-slide summary

Making Two Cameras Agree: EO/IR Fusion & Self-Calibration

A plain-language explainer of why our two-camera drone detector wasn't fusing, what we found, and how we fixed it so it now calibrates itself.

0. The 30-second version

We point **two different cameras** at the sky to catch drones: a normal daylight camera (**EO**) and a thermal/heat camera (**IR**). Each camera has its own AI detector. The plan was to **combine** ("fuse") their answers so that when *both* cameras see the drone, we're more confident, and when one is blinded (e.g. EO at night), the other carries it.

It wasn't fusing. We traced it to one cause: **the two cameras don't point at exactly the same patch of sky**, and they have **different zoom**, so the drone shows up in a *different place* in each image. The combiner only merges two detections if they sit on top of each other — and they never did.

The fix is **registration**: a one-time mathematical "translation" between the two cameras' coordinate systems. Once we apply it, the two detections line up, and fusion works on ~90% of frames. Better still, the system can now **figure out that translation by itself**, just by watching the drone fly.

1. Vocabulary (read this first)

Term	What it means, plainly
EO	"Electro-Optical" — a normal visible-light camera (what your eye sees). Great by day.
IR	"Infrared" — a thermal camera that sees heat. Works in darkness.
Detector / model	The AI that looks at one image and draws a box around the drone.
Bounding box	The rectangle the detector draws around the target. Four numbers: left, top, right, bottom (in pixels).
Confidence	How sure the detector is, from 0 to 1 (e.g. 0.75 = "75% sure that's a drone").
Fusion	Combining the EO answer and the IR answer into a single, more trustworthy answer.
IoU	"Intersection over Union" — a 0-to-1 score for <i>how much two boxes overlap</i> . The core test for "are these two boxes pointing at the same thing?"
Registration	Lining up two images/cameras into one shared coordinate system, so "position (x, y)" means the same thing in both.
Affine transform	A simple, fixed recipe to convert positions from one camera into the other's — it allows shift, zoom, stretch, and rotation. Six numbers.
Tracking	Following the <i>same</i> drone across frames over time, giving it a stable ID and a velocity. Filters out one-frame false alarms.
Co-boresighted	Engineering term for "two cameras aimed at exactly the same scene, pixel-for-pixel." Our cameras are not this.

2. The setup and what each piece did

- **Inputs:** two video files of the same drone flight — `EO-stream.mp4` (4K, 3840×2160) and `IR-stream.mp4` (1280×1024). Both 775 frames.
- **Detectors:** one trained on EO, one trained on IR.

First finding — the detectors are excellent. Run on their own, each found the drone in **all 775 frames**:

Camera	Frames with the drone found	Typical confidence
EO	775 / 775	~0.55–0.60
IR	775 / 775	~0.72–0.77

So the AI was never the problem. The problem was getting the two answers to agree.

3. How "do these two boxes match?" is measured: IoU

Fusion has to decide whether the EO box and the IR box are looking at the **same object**. It uses **IoU — Intersection over Union**:

$$\text{IoU} = \frac{\text{area where the two boxes OVERLAP}}{\text{total area the two boxes COVER together}}$$

- IoU = **1.0** → the boxes are identical.
- IoU = **0.5** → they overlap by half.
- IoU = **0** → they don't touch at all.

The combiner's rule: if IoU is ≥ 0.40 , call it the same object and fuse them.

A real example from our footage (frame 0)

Both boxes are written as `[left, top, right, bottom]` in pixels, in the IR image:

IR box (the drone, per IR)	:	[139, 801, 210, 841]	
EO box, just resized to match	:	[308, 816, 364, 848]	→ IoU = 0.00
EO box, after registration	:	[133, 805, 217, 839]	→ IoU = 0.74

- **Before registration:** the EO box *starts* at x=308, but the IR box *ends* at x=210. There's a 98-pixel gap between them — they don't touch. **IoU = 0** → no fusion. This happened on essentially every frame.
- **After registration:** the EO box `[133, 805, 217, 839]` lands almost exactly on the IR box `[139, 801, 210, 841]`. **IoU = 0.74** → well past 0.40 → they fuse.

That single jump from 0 to 0.74 is the whole story.

4. Why it was failing: the cameras don't share a coordinate system

We measured *where* the drone appeared in each camera, frame by frame. The horizontal gap between the two boxes was not fixed — **it wandered from -185 pixels to +163 pixels and even changed sign** (sometimes EO was left of IR, sometimes right). The drone is only ~58 pixels wide, so a gap of 185 is three whole drone-widths off.

Why does the gap change? Because the two cameras have different *zoom* and are aimed slightly differently. As the drone moves across the sky, the mismatch between the two views changes with it. This is normal for two separate cameras — it's called *parallax / field-of-view difference*.

The trap we ruled out

A natural first instinct is "just resize or crop one image to match the other." **This cannot work, for a precise mathematical reason: IoU is unaffected by resizing.** If you shrink both boxes by the same amount, the overlap *ratio* stays identical. Resizing changes how big things look, not whether two boxes sit on top of each other. We

confirmed this empirically — scaling the 4K EO down to the IR size left IoU at **exactly 0.000**.

5. The fix: one fixed "translation" between the cameras (registration)

If the relationship between the two cameras is *consistent*, we can capture it once as a fixed formula and apply it to every EO box to move it into the IR image's coordinates. We tested four candidate formulas, from simplest to richest, and measured the **average leftover error** (how far off the converted EO box is from the true IR box). Lower is better; "good" means well under the 58-pixel drone width:

Formula	What it can do	Leftover error (median)
Translation (a shift / crop)	slide left-right, up-down	97 px ❌
Similarity	shift + zoom + rotate (uniform)	21 px
Affine	shift + zoom + stretch + rotate	4.7 px ✅
Homography	affine + perspective	4.2 px ✅

Translation (i.e. a crop/shift) fails badly — 97 px, confirming the instinct from §4. **Affine nails it at ~5 px**. The reason affine succeeds where a shift fails is visible in the fitted numbers:

horizontal zoom ≈ 1.48 vertical zoom ≈ 1.06

The two cameras differ in zoom **and** their horizontal vs vertical zoom differ from each other (1.48 vs 1.06) — the EO image is stretched relative to IR. A crop only slides; it can't zoom or stretch, so it can never represent this. An affine can. **One affine, fixed for the whole flight, is all it takes.**

Why one fixed formula is even valid

The drone is far away (against the open sky). For distant objects, the relationship between two cameras is a single fixed transform that does **not** depend on where the object is — so the affine we fit from one stretch of the flight applies to the entire flight. The proof: ~5-pixel error across all 775 frames, covering the whole field of view.

6. Results after registration

Setting	Frames that fused (out of 775)
No registration (original)	~8 (only when the drone happened to drift through the one spot where the views crossed)
With registration	~700 (~90%) , with stable tracking

The remaining ~10% are frames where the drone is briefly hard to see or the leftover error peaks; tracking bridges most of those gaps anyway.

7. The clever part: it calibrates *itself*

We don't want an engineer measuring cameras with a checkerboard. The system derives the affine **on its own, just by watching detections**:

1. **Free, certain matches.** On any frame where EO sees **exactly one** drone and IR sees **exactly one** drone, they *must* be the same object — no formula needed to know that. So we record "EO saw it here → IR saw it there." We got 756 such free matches from this one flight.
2. **Wait for the drone to roam.** You cannot work out zoom/stretch from a drone that's sitting still — all the matches would be piled in one spot. The system measures how much the drone has *spread across the frame* and refuses to commit until it has seen enough variety. (In our clip the drone hovered for the first ~13 seconds, so it waited — correctly — until it started moving.)
3. **Fit and lock.** Once there's enough spread, it computes the affine, rejecting any odd bad match (e.g. a bird mistaken for the drone) with a standard robust-fitting

method (RANSAC). If the fit is tight (~2 px here), it **locks**.

4. **Keep improving.** As the drone explores more of the sky, it refines the affine toward the best possible version.

What this looked like live: the system locked the transform automatically at frame 387 (the moment the drone had roamed enough), at ~2 px accuracy, with no human input.

The practical recipe for a fixed installation

Because your cameras don't move, you calibrate **once** and reuse it forever:

- **Calibrate once:** run on any clip where the drone roams; save the affine (`--save-reg matrix.npy`).
- **Deploy:** load that saved affine (`--reg-matrix matrix.npy`). Now there's **zero warm-up** — fusion works from the very first frame. In this mode we fused **697/775 frames (90%)** and tracked **745/775**.

You'd only ever recalibrate if someone changes the **zoom** or **physically moves** a camera.

8. Honest limitations (what to watch for)

Limitation	Plain meaning
One transform per camera setup	Change the zoom or bump a camera → the saved affine is stale; recalibrate.
Far targets only	The "one fixed formula" trick relies on targets being far away. A drone very close to the cameras would need true 3-D handling.
Tested in daylight only	Every frame here was bright (daytime). The night-time behaviour (where IR should take over) is built but not yet validated on real night footage.
One drone at a time	We validated with a single target. Multiple drones/birds at once need extra logic to avoid mixing them up.

Limitation	Plain meaning
Self-calibration needs motion	If the drone never roams across the frame, the system can't learn the stretch — it stays in warm-up. A short calibration flight solves this.
Camera frame-rates differ slightly	EO runs at 29.97/s, IR at 30.00/s. Over long runs they drift apart in time; pairing should eventually be done by timestamp, not frame number.

9. One-slide summary

- Two cameras, two good detectors — but their images **don't line up**, so the combiner saw "two different things" and refused to fuse.
- The misalignment is a **zoom + stretch**, not a simple shift — so cropping can never fix it, but **one affine transform** does (~5 px).
- With it, fusion jumps from **~8** → **~700 frames** out of 775.
- The system now **calibrates itself** by watching the drone fly, and for a fixed rig you **calibrate once and reuse**.